

CSCE 633 MACHINE LEARNING · MODULE 06 · LECTURE NOTES

Trees and Ensembles

What actually wins on tabular data.

MODULE OUTCOMES

- Explain how a decision tree is grown and why it overfits.
- Explain bagging and what it reduces.
- Explain boosting and what it reduces.
- Contrast random forests with gradient boosting.
- Interpret feature importance correctly, including its traps.

1 A single tree

TREES SPLIT GREEDILY ON THE BEST SINGLE QUESTION

At each node, consider every feature and every candidate threshold, and choose the split that most reduces impurity — Gini or entropy for classification, variance for regression. The search is exhaustive over splits and greedy over depth.

Then recurse on each side until a stopping rule fires: minimum samples per leaf, maximum depth, minimum impurity decrease, or no split improving anything.

A fully grown tree fits the training data exactly — every leaf can be made pure — **and generalises badly**. It is the textbook high-variance model of Module 03 §1.

And it is unstable. Changing a single training point can change an early split, which changes every subtree beneath it, producing a completely different tree that performs about as well. **That instability looks like a flaw and is exactly what makes averaging effective** (§2) — averaging helps most when the individual models' errors are uncorrelated, and unstable models trained on resampled data produce exactly that.

Trees are good at	Trees are bad at
Mixed numeric and categorical features in one model, with no encoding required.	Smooth or linear relationships, which must be approximated by a staircase of splits — inefficiently.
Requiring no feature scaling whatsoever. Splits are invariant to any monotonic transformation.	Extrapolating beyond the training range. A tree's prediction is constant outside the data it saw, which is sometimes safe and sometimes badly wrong.
Missing values, handled natively by surrogate splits or by learning a default direction.	High variance when used alone (§1).

Trees are good at	Trees are bad at
Finding interactions automatically — a split below another split is an interaction, discovered rather than specified.	Diagonal decision boundaries. Axis-aligned splits approximate a diagonal with a staircase, which costs depth and samples.
Being invariant to monotonic transforms of any feature.	Very wide data with many irrelevant features , where the greedy search finds spurious splits.

No scaling, native missing-value handling, and automatic interaction discovery together mean a tree ensemble needs dramatically less preprocessing than any other model family — which is a substantial and underrated part of why they dominate practical tabular work (§3).

2 Bagging and random forests

BAGGING REDUCES VARIANCE AND LEAVES BIAS ALONE

Train many trees on bootstrap samples of the data and average their predictions. Averaging n independent estimates of the same quantity divides the variance by n while leaving the bias unchanged — which is exactly the right medicine for a high-variance, low-bias model.

But the trees are not independent. They see heavily overlapping bootstrap samples, and more importantly they will all discover and use the same few strong features, so their errors correlate and the variance reduction falls short of $1/n$.

So a random forest also samples the features considered at each split — typically $\sqrt{d}$ of them for classification. **This is the key addition over plain bagging:** it forces different trees to use different features, decorrelating them and recovering much of the lost variance reduction.

And the out-of-bag samples provide a free validation estimate. Each bootstrap sample omits about a third of the data, so every training point can be scored by the subset of trees that did not see it — giving a cross-validation-like estimate at no extra training cost, which is a genuinely elegant property.

3 Boosting

```

BAGGING    many DEEP trees, in PARALLEL, AVERAGED
           each is a full model; averaging cancels variance

BOOSTING   many SHALLOW trees, in SEQUENCE, SUMMED
           F_0 = constant (the mean)
           for m = 1..M:
               r = negative gradient of the loss at F_{m-1}
                   (for squared loss, just the residual)
               fit a shallow tree h_m to r
               F_m = F_{m-1} + eta * h_m           <- eta = learning rate

           Each tree corrects what the ensemble so far got wrong.
           Summing weak learners REDUCES BIAS.
           Averaging strong learners REDUCES VARIANCE.

THREE KNOBS, which interact:
  n_estimators    more -> lower bias, eventually overfits
  learning_rate   smaller -> needs more trees, generalises better
  max_depth       3-8; deep trees in boosting overfit quickly

Lower the learning rate and raise the tree count TOGETHER.

```

Bagging reduces variance; boosting reduces bias. They are not two flavours of one idea but opposite responses to the two terms of Module 03's decomposition, and the model each starts from reflects that: bagging begins with a model that is too flexible and calms it down, boosting begins with models that are far too simple and accumulates them.

WHY GRADIENT-BOOSTED TREES WIN ON TABULAR DATA

Tabular features are heterogeneous — different units, wildly different scales, a mixture of continuous, ordinal, and categorical types, with no meaningful notion of distance between rows. **Trees handle all of that natively; neural networks require it to be engineered away first.**

Relationships in tabular data are frequently piecewise and non-smooth — thresholds, eligibility cut-offs, category effects, step changes at round numbers. **Trees represent a step exactly with one split;** a smooth function approximator spends capacity approximating it.

And the datasets are small by deep learning standards. A few thousand to a few million rows, where boosting is extremely sample-efficient and networks are still in their data-hungry regime.

Most fundamentally, there is no useful inductive bias to exploit. Convolution wins on images because it assumes spatial locality and translation invariance, both of which are true of images; attention wins on text because it assumes sequence structure. **A spreadsheet has neither property,** and column order is arbitrary — so the architectural advantage that makes deep learning dominant elsewhere simply does not apply. **This has been confirmed repeatedly in benchmark studies,** and it is the clearest available demonstration that architecture is about matching structure in the data rather than about capacity.

4 Feature importance

Method	What it measures	The trap
Impurity (Gini) importance	Total impurity reduction attributable to splits on that feature. The default, and nearly free.	Systematically biased toward high-cardinality features — a continuous feature or an ID-like column offers many candidate splits and will score highly even when random. Widely unknown and genuinely misleading.
Permutation importance	The drop in validation score when that feature's values are shuffled. Measured on held-out data, so it reflects generalisation.	Correlated features share the credit and both look weak — shuffling one leaves the other to carry the signal, so neither appears necessary.
SHAP values	A per-prediction, additively fair attribution derived from cooperative game theory.	Expensive to compute , and still a statement about correlation within the model, not about causation — which the presentation of SHAP plots tends to obscure.

IMPORTANCE IS ABOUT THE MODEL, NOT THE WORLD

Feature importance reports what the model used. It does not report what causes the outcome, and the two are routinely conflated in exactly the settings where the distinction matters most.

A proxy will look important. If postcode predicts income and income affects the outcome, postcode will score highly — and changing someone's postcode would do nothing. **The model is right about the correlation and the causal reading is wrong.**

And correlated features split the credit, so two features carrying the same underlying signal each appear about half as important as that signal really is — which can make a genuinely important quantity look unimportant twice over.

So treat importance as a debugging tool rather than a finding. Its highest-value use is spotting leakage (Module 12 §1): **an implausibly dominant single feature is leakage far more often than it is a discovery**, and checking the top of the importance list is the cheapest leakage audit available. **Do not use it to make claims about the world** — that requires a causal design, which is a different subject.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Hastie et al. — The Elements of Statistical Learning , chapters 9, 10 and 15 (free PDF)	Trees, boosting, and random forests, with the bias-versus-variance framing of §2 and §3 made precise.

Resource	What it covers for this module
Friedman — Greedy Function Approximation: A Gradient Boosting Machine (free)	The §3 algorithm, from its author. The gradient framing is what makes it general.
Grinsztajn, Oyallon & Varoquaux — Why do tree-based models still outperform deep learning on tabular data? (free)	The §3 callout, measured carefully across many datasets, with the reasons isolated experimentally.
Strobl et al. — Bias in random forest variable importance measures (free)	The high-cardinality bias of §4 , demonstrated. Read before reporting a Gini importance ranking.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a decision tree with Gini splitting from scratch.
- 2 **Demonstrate instability:** change one training point and show the tree structure changes substantially.
- 3 **Fit a tree to a diagonal boundary** and visualise the staircase. Measure how depth affects the approximation.
- 4 Implement bagging and plot test error against the number of trees.
- 5 **Add feature subsampling** and show the extra variance reduction.
- 6 Compute out-of-bag error and compare it against a held-out estimate.
- 7 Implement gradient boosting for squared loss from scratch.
- 8 **Sweep learning rate against tree count** and plot the surface. Confirm the trade.
- 9 **Compare boosted trees against a neural network** on a tabular dataset, at equal tuning effort.
- 10 **Add a random high-cardinality ID column** and show it ranks highly in Gini importance and not in permutation importance.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How is a tree grown, and why is it unstable?
- 2 Give five things trees are good at and five they are bad at.
- 3 What does bagging reduce, and what does feature subsampling add?
- 4 What is out-of-bag error and why is it free?
- 5 What does boosting reduce, and how does it differ from bagging?

- 6 Name the three boosting knobs and the reliable tuning move.
- 7 Give four reasons boosted trees win on tabular data.
- 8 Compare three importance methods and the trap in each.
- 9 Why is importance not causal, and what is it good for?